SOMA: Signed Posts as Hyperedges

Category, ontology and convergence in a network of public posts

CoSys Work Pvt. Ltd.
cosys.work

November 2026

Abstract

SOMA is a local-first application for public posts. A person writes once, on their own device, under a pseudonym, and the post reaches several networks. Each network gets its own copy, and the one original is a record the author signed. This paper describes the mathematics the design rests on. We model the record as a hypergraph and read it through its Levi graph, which gives a category whose morphisms are typed references. We treat each network's copy of a post as the image of a structure-preserving map that remembers where it came from. We state the laws a bridge between two protocols must satisfy, the first being that importing what was exported is the identity. We place the semantics in three layers: what a person wrote, what a machine read into it, and what is computed over. We argue that trust can be published as a relation and never as a number. Finally, we show how replicas converge without coordination, how a private circle's history is partitioned by key epochs, and how a post can be shown to have existed by a point in public time without trusting anyone who holds it.

1 The problem, and three commitments

A person who writes in public today writes into several networks, each with its own account, its own copy and its own rules for what a post may be. The copies drift. A link is shortened in one place and a line break dropped in another, and nothing ties them back to what the author actually wrote. SOMA starts from the opposite end. There is one original, signed on the author's device, and every other appearance of the post is a *projection* of that original that says what it is a copy of.

The design rests on three commitments.

1. **The original is a signed record, kept by its author.** Devices hold their own records and work offline. Servers relay and store copies, and none of them is the source of truth.

2. **Pseudonymity toward the network, accountability toward the operator.** The network sees a key and the assertions others have signed about it. Whoever admitted the person knows who they are, and that knowledge never enters the record.

3. **No scores.** Nothing in the published record is a number about a person: no rank, no reputation, no count that stands in for trust. Order is computed by each reader, for that reader, from relations that are public.

2 The record as a hypergraph

2.1 Records, incidences and roles

Definition 2.1. *A* record *is a tuple $r = (a, t, k, I, c)$: an author key a, a time t, a kind k drawn from a registered vocabulary, a finite list of* incidences $I = \big((\rho_1, x_1), \dots, (\rho_n, x_n)\big)$*, and a content c. Each incidence pairs a* role ρ_i*, drawn from a closed vocabulary, with a target x_i, which is another record or a key. The record's identifier is $\mathsf{H}(a, t, k, I, c)$ for a collision-resistant hash H, and the author's signature covers that identifier.*

A single record often states one fact about several things at once. A reply names the post it answers and the person it addresses; a vouch names who vouches and for whom; a deletion request names every version it withdraws. Such a record is a *hyperedge*.

Definition 2.2. *The* record hypergraph *is $\mathcal{H} = (V, E, \iota)$, where E is the set of records, V is the set of everything a record can name (records and keys), and $\iota \subseteq E \times \mathrm{Roles} \times V$ is the set of role-labelled incidences. Its* Levi graph *$L(\mathcal{H})$ is the bipartite graph with one node per record, one node per vertex, and one edge, labelled ρ, for each $(e, \rho, v) \in \iota$.*

The Levi graph is how the record is stored and indexed. Each record is already a node, and its incidence list, written once with its role labels, is its out-edges. A query such as "every record that names x in role ρ" is the in-neighbourhood of x in $L(\mathcal{H})$, and needs no knowledge of what kind of record did the naming.

2.2 The category of records

Proposition 2.3. *The reference graph on records is acyclic.*

Proof. A record's identifier is the hash of its incidences, so a record can name only identifiers that exist when it is made. A cycle $r_1 \to r_2 \to \cdots \to r_m \to r_1$ would require each $\mathsf{H}(r_i)$ to be computed from a value that already contains $\mathsf{H}(r_{i+1})$, and so $\mathsf{H}(r_1)$ from a value containing itself. That is a fixed point of H composed with the encoding, which collision resistance makes infeasible to find. □

Definition 2.4. *Let* **Rec** *be the free category on the directed, role-labelled reference graph: objects are records and keys, and a morphism $r \to x$ is a path of references from r to x, with its sequence of roles. Composition is concatenation of paths, and identities are the empty paths.*

By Proposition 2.3, **Rec** has no non-identity endomorphisms, and the relation "r depends on x" (there is a morphism $r \to x$) is a partial order. Questions people actually ask become statements about **Rec**. *Where did this number come from* is the set of morphisms from a computed record to the records it read. *What does withdrawing this post affect* is the set of morphisms into it. A feature of the application is a family of maps between kinds of record, and the features available to a given record type form a hom-set. All of this reads the stored graph and stores nothing new.

2.3 Versions and posts

An author may revise a post. A revision is a new record with a *parent* incidence to the version it was made from and a *post* incidence to the post's first version. A revision made on two devices from the same parent produces a fork.

Definition 2.5. *For a first version p, the* version set V_p *is the set of records whose post incidence is p, together with p. The parent incidences make V_p a rooted tree. The* latest version *is the maximum of V_p under the total order (t, H): time first, then identifier.*

The order is total and computed from the records alone, so every replica that holds the same V_p shows the same latest version, with no coordination. A *remix*, whether of one's own post as a new post or of someone else's, is a new root that keeps a parent incidence for attribution and no post incidence, so it starts a version tree of its own. A post is its version tree, and readers see the tree's maximum.

3 Projections as structure-preserving maps

3.1 Networks as categories

Every network has posts and relations between them: a reply answers a post, and a quote or a reshare refers to one. Write $\mathbf{N}_k$ for network k seen this way: objects are its posts, and morphisms are generated by its reply and quote relations. Write **Post** for the same structure over SOMA's originals, a subcategory of **Rec**.

Definition 3.1. *A* projection *to network k is a functor $P_k \colon \mathbf{Post} \rightharpoonup \mathbf{N}_k$, defined on the posts that network can carry. It sends a post to its rendering within the network's limits on length, media and threads, and it sends a reply to a reply.*

A projection may be lossy. A long post becomes a thread, and a file the network cannot hold becomes a link. A projection must keep *where it came from*.

Definition 3.2. *An* origin map origin_k *sends a copy in $\mathbf{N}_k$ that names an original to that original.*

Proposition 3.3. *If every projected copy names its original, then $\mathsf{origin}_k \circ P_k = \mathrm{id}$ on the domain of P_k, so P_k is a split monomorphism there.*

The proof is immediate from the definitions. However far a copy travels, the network it reaches can always take it back to exactly one original, and two copies of one original can be recognised as the same post.

3.2 Matching a copy that a platform changed

Some networks give no way to name an original. There, the author shares the copy by hand and later links it back, and the platform may have changed the text in transit. Let T be the set of texts and G the monoid generated by the edits platforms are known to make: shortening a link, changing line endings, collapsing whitespace, inserting invisible characters.

Definition 3.4. *A* normalisation *is a function $\nu \colon T \to T$ with $\nu \circ g = \nu$ for every $g \in G$. A copy u* matches *an original s when $\nu(u) = \nu(s)$.*

A normalisation is a map onto a set of representatives for a quotient coarser than the orbits of G. The choice is a trade-off. A finer ν (exact match) rejects honest copies, and a coarser one accepts copies that differ in substance. SOMA uses one rule for every network: canonical Unicode composition, unified line endings, runs of whitespace collapsed, invisible characters removed, and every link replaced by a single token, since links are what platforms rewrite most. Rules for particular networks can refine the quotient later without changing what a match means.

4 Ontology: the sign, the reading and the token

4.1 Three layers

SOMA keeps three things apart, after the semiotic distinction between a sign, its interpretation, and the object it stands for.

The sign is what a person wrote, photographed or signed. It is stored as made and never rewritten.

A reading is an interpretation of a sign, made by a model, a bridge or a person. It is its own record, with an incidence in the role *reads* to the sign it interprets. A wrong reading is corrected by a new reading, and the sign is never touched.

The token is the typed incidence the machine computes over: the Levi graph's edges.

Any value the application shows can be followed back along a morphism in **Rec** from a token, through the readings, to a sign, so every computed figure has a source.

4.2 Assertions: hyperedges about hyperedges

Definition 4.1. *An* assertion *is a record whose kind is a* claim *from a closed vocabulary, whose incidences name its subjects, and whose author is the party entitled to make that claim.*

An assertion may name other assertions, so the record is a hypergraph whose hyperedges can be vertices of further hyperedges. Only the key that signs an assertion decides whether its author may make that claim. For a public network the relevant claims are few: that an operator admitted a key, that a key vouches for another, and that a key withdrew either.

4.3 Trust as a relation

Definition 4.2. *A* vouch *is an assertion $v(a, b)$: key a vouches for key b. It carries no number, no reason, and no weight. Withdrawing it is a later record that deletes it.*

The published trust structure is therefore a relation $R \subseteq K \times K$, the set of vouches that stand. A reader who belongs to a set of circles C computes, for themselves alone,

$$d(b) \;=\; \min\{\, n \;:\; b \in C^{(n)} \,\}, \qquad C^{(0)} = \bigcup C, \quad C^{(n+1)} = C^{(n)} \cup R[C^{(n)}],$$

the number of vouching steps from the reader's own circles to b, capped at a small depth. Their feed is ordered lexicographically by $(d, -t)$: nearer first, newer first within each distance.

Proposition 4.3. *Nothing a reader can observe of another reader's ordering is published.*

Proof. The ordering is a function of the public relation R and of the reader's own circles C. The circles' memberships never leave the circles. The ordering itself is not a record and is never signed or sent. □

Distance is ordinal and private. No global quantity exists for anyone to buy or game, and no number about a person is published.

5 Bridges and their laws

A bridge is a peer with two faces. Toward a foreign protocol it receives items as untrusted input; toward SOMA's network it writes records that name where each came from. Between the two sits a *codec*.

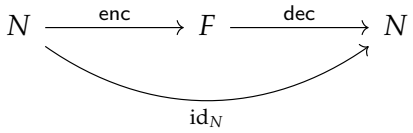
5.1 The codec law

Definition 5.1. *A* codec *between native records N and foreign objects F is a pair of partial maps* $\mathsf{enc}\colon N \rightharpoonup F$ *and* $\mathsf{dec}\colon F \rightharpoonup N$.

Definition 5.2 (The codec law)**.** *For every native record n in the domain of* enc,

$$\mathsf{dec}(\mathsf{enc}(n)) = n.$$

The law makes enc a section of dec, so enc is a split monomorphism and dec a split epimorphism onto its image.



The converse, $\mathsf{enc} \circ \mathsf{dec} = \mathrm{id}_F$, does not hold and is not asked for. Foreign objects carry fields the native side has no place for, and they come back as a native record that names its foreign origin. The law has a second half for those records. Exporting a record that is itself a copy of a foreign object does not produce a new foreign object; it produces a *reference* to the original. Writing ι_F for the inclusion of foreign originals by reference,

$$\mathsf{enc}(\mathsf{dec}(f)) = \iota_F(f) \quad \text{for every foreign } f \text{ in the domain of } \mathsf{dec}.$$

Together the two halves say a post can travel out through a bridge and back again, in either direction, without ever becoming a second post.

The first half is only checkable because the exported object carries enough to rebuild the original exactly, words and incidences, and because dec rebuilds it and recomputes its identifier before accepting it as the original. A foreign object that claims to be a copy of an original but does not rebuild to that identifier is read as a stranger's post, so a false origin gains nothing.

Remark 5.3. A codec satisfying both halves is a well-behaved lens in the sense of bidirectional programming, with dec as *get* and enc as *put* from the empty source. The codec law is PutGet, and the reference half plays the part of GetPut for objects that originate on the foreign side.

5.2 Idempotence by origin

A bridge sees the same foreign object more than once, through retries, redeliveries and second copies. Every record it writes is keyed by the origin it names.

Proposition 5.4. *Let S be the bridge's state, a map from origins to the records written for them, and let* $\mathsf{run}(S, X)$ *write, for each received item $x \in X$, the records* $\mathsf{dec}(x)$ *whose origins are not in S. Then* $\mathsf{run}(\mathsf{run}(S, X), X) = \mathsf{run}(S, X)$*, and delivering X twice writes what delivering it once writes.*

An item that cannot be written yet, because the account it is for has not paired, is held and retried. Holding has a deadline. An item still waiting at the end of it is refused, because it is someone's data held by a bridge nobody asked to hold it.

5.3 Verification before reading

Every request a bridge receives is authenticated before any of its content is interpreted: the body must hash to the digest the request states, the signature must cover the request line, the host, the date and that digest, the date must fall within a window, and the signer must be the actor the request speaks for. A request that fails any check is discarded unread.

6 Convergence, epochs and proof

6.1 The store as a join-semilattice

Definition 6.1. *A replica's state is a finite set of signed records. Merging two states is union.*

Sets under union form a join-semilattice: union is associative, commutative and idempotent. A state that only grows by union is a state-based conflict-free replicated data type, so replicas that have received the same records are equal, whatever order or path the records took. Deletion does not break this, because a deletion is itself a record. What a reader sees is a deterministic function $\mathsf{view}(S)$ of the set: the latest version of each post, less those withdrawn. Equal sets give equal views.

Remark 6.2. The set only grows, while view can shrink: a withdrawal removes a post from it. Convergence needs only that view is deterministic, so each replica computes the same view from the same set.

6.2 Reconciling two replicas

Two replicas bring their sets together by *set reconciliation*. They compare fingerprints of ranges of their records, ordered by time and identifier, and split any range whose fingerprints differ until the differing records are named. The communication grows with the size of the difference rather than the size of the sets. Each result carries a *watermark*, saying where it came from, when and how far it reached, so the application states how fresh a view is and never assumes it is complete.

Between devices that are online together, records spread by epidemic broadcast: a membership layer keeps each device in contact with a few others, and a broadcast tree carries each record once along it. Every hop checks a record's signature and kind before passing it on and drops anything already seen.

6.3 Circles and key epochs

A circle is a small group whose posts are readable by its members only. It holds a sequence of symmetric keys $\kappa_0, \kappa_1, \ldots$, one per *epoch*, and a membership function $m(j)$, the set of members during epoch j. Each circle post is sealed under the key of the epoch it was written in, with the circle, the epoch and the author bound into the authenticated data.

Definition 6.3. *Adding a member shares the current key. Removing a member b in epoch j starts epoch $j+1$ at time τ_{j+1}, with a fresh key κ_{j+1} shared only with $m(j+1) = m(j) \setminus \{b\}$.*

Proposition 6.4. *A member removed at the start of epoch $j+1$ cannot read a post sealed in any epoch $i > j$. A post sealed in epoch $i \le j$ is accepted only if dated before τ_{i+1}.*

Proof. The removed member holds $\kappa_0, \ldots, \kappa_j$ and no later key, and authenticated encryption gives no means to open a post sealed under a key one does not hold. The second statement is the acceptance rule. □

The second clause stops a removed member from writing into the circle after leaving under a key they still hold. It cannot stop them from back-dating such a post to before their removal. That limit is shared by every scheme without a trusted clock.

6.4 Proof of existence

To show that a version existed by some point in public time, a device commits a batch of versions to one value. Each leaf is the hash of a version's identifier together with a random nonce, so no leaf reveals which version it is, and the leaves form a Merkle tree with root ρ. The root is submitted to independent timestamping services, which aggregate many roots and in time commit their aggregate to a public blockchain.

Proposition 6.5. *Given the version, its nonce, its path to ρ, and the path from ρ to a block, anyone can check that the version existed by the time of that block, without trusting the author, SOMA or the timestamping services.*

The proof only speaks for the version it was made for. A path proves membership of one leaf, and the nonces keep a batch's other members private. The statement is about existence by a time. It is strong evidence of authorship and timing, and it is not a registration of rights.

6.5 Bounded extensions

SOMA runs extensions written by others, such as renderers for sound and documents and views of conversations. Each is a pure function of an event and of the projections the reader chose to give it, run in an interpreter with a fixed allowance of steps and memory.

Definition 6.6. *An extension's* manifest *names the host operations it may call, a subset M of the host's interface, and the kinds of record it may read and suggest. An extension that asks for anything outside M is refused before it runs.*

With a step allowance, every call is a total function onto outputs extended with a failure value: it returns an output, or it is stopped and returns nothing. Two interpreters of the same extension are held to the same behaviour on a fixed set of inputs. That is extensional equality on a finite test domain, and it is what lets the same extension run on every surface.

7 What we do not claim

- **Anonymity toward the operator.** The operator that admitted a person can know who they are. The network cannot learn it from the record.
- **That deletion reaches every copy.** A withdrawal is a record that honest replicas apply; a copy someone else already kept may remain, and the application says so.
- **That a network's copy cannot drift.** Projections can be edited where they land. What we claim is that the original stays checkable and every copy can be traced to it.
- **A trusted clock inside circles.** Epoch acceptance bounds what a removed member can do, and back-dating remains possible, as above.

8 Conclusion

A signed record is a hyperedge, and its references give a category. A copy elsewhere is a functor's image that remembers its source. A bridge is a pair of maps bound by one law. Trust is a relation, and order is each reader's own computation. Replicas converge because their state is a semilattice, and proof of existence is a path in a hash tree. Each of these can be stated as a law, and the system has a test for each.

References

[1] F. W. Levi. *Finite Geometrical Systems*. University of Calcutta, 1942.

[2] C. Berge. *Hypergraphs: Combinatorics of Finite Sets*. North-Holland, 1989.

[3] S. Mac Lane. *Categories for the Working Mathematician*. Springer, 2nd ed., 1998.

[4] D. I. Spivak and R. E. Kent. Ologs: a categorical framework for knowledge representation. *PLoS ONE* 7(1), 2012.

[5] C. S. Peirce. *Collected Papers*, vol. 2: Elements of Logic. Harvard University Press, 1932.

[6] J. N. Foster, M. B. Greenwald, J. T. Moore, B. C. Pierce and A. Schmitt. Combinators for bidirectional tree transformations. *ACM TOPLAS* 29(3), 2007.

[7] M. Shapiro, N. Preguiça, C. Baquero and M. Zawirski. Conflict-free replicated data types. In *SSS*, 2011.

[8] A. Meyer. Range-based set reconciliation. In *SRDS*, 2023.

[9] A. Demers et al. Epidemic algorithms for replicated database maintenance. In *PODC*, 1987.

[10] J. Leitão, J. Pereira and L. Rodrigues. HyParView: a membership protocol for reliable gossip-based broadcast. In *DSN*, 2007; and Epidemic broadcast trees, in *SRDS*, 2007.

[11] R. C. Merkle. A digital signature based on a conventional encryption function. In *CRYPTO*, 1987.

[12] S. Haber and W. S. Stornetta. How to time-stamp a digital document. *Journal of Cryptology* 3(2), 1991.

© 2026 CoSys Work Pvt. Ltd. This work is licensed under the Creative Commons Attribution 4.0 International licence (CC BY 4.0), `https://creativecommons.org/licenses/by/4.0/`. You may share and adapt it, for any purpose, provided you give credit.